

Projet INFO-H-3000 : Planification Équitable des Tournois Round Robin avec Méta-heuristiques

Nicholas Birch de la Calle
Gabriel Badan
Milena Paques

Enseignant : Yves De Smet
Assistant : Jean Rosenfeld

Mai 2025

Table des matières

1 Introduction

1.1 Contexte du Projet

La planification de tournois "Round Robin" simples (SRR), où chaque participant affronte tous les autres une unique fois, est un défi classique. Au-delà de la simple organisation des rencontres, l'équité du calendrier est primordiale. Des facteurs comme jouer à domicile ou avec les pièces blanches aux échecs introduisent des avantages qui, s'ils sont mal répartis, peuvent fausser les résultats. Ce projet vise à élaborer des calendriers SRR optimisant l'équité, en tenant compte de la force relative des adversaires et de la régularité des séquences domicile/extérieur.

1.2 Objectifs et Approche

Notre objectif est de générer des calendriers SRR équitables pour n joueurs, en minimisant les déséquilibres liés à :

- La **HomeStrength** : l'avantage de jouer à domicile contre des adversaires de force variable.
- La **Pénalité de séquence** : les séries de matchs consécutifs à domicile ou à l'extérieur.
- La **Max Deviation** : l'écart maximal du nombre de matchs à domicile par joueur par rapport à l'idéal.

Pour les n impairs, un joueur fictif est introduit pour gérer les journées de repos ("bye"). Nous avons exploré une méthode exacte (Programmation Linéaire en Nombres Entiers Mixtes, MILP) pour les petites instances et une méta-heuristique de Recuit Simulé (SA) pour les instances plus grandes, en mettant l'accent sur l'optimisation de cette dernière pour une analyse approfondie.

1.3 Structure du Rapport

Ce rapport détaille d'abord la modélisation mathématique du problème (Section ??). Il aborde ensuite la normalisation des métriques et la fonction objectif (Section ??), puis la calibration des poids de cette fonction (Section ??). La Section ?? présente notre stratégie de développement du Recuit Simulé, les optimisations implémentées, une comparaison de performance détaillée, et une analyse qualitative des plannings obtenus. Enfin, la Section ?? résume nos travaux et propose des pistes pour l'exploration future.

2 Modélisation Mathématique

2.1 Définition du Problème

Le problème consiste à générer un calendrier pour un tournoi Round Robin simple (SRR) impliquant n joueurs, où n peut être pair ou impair. Les joueurs sont numérotés de 1 à n , et cet indice représente également leur classement de force (1 étant le plus fort, n le plus faible). Si n est impair, un joueur fictif numéroté $n + 1$ est ajouté, portant le nombre total de participants à $n_{eff} = n + 1$. Le tournoi se déroule sur $n_{eff} - 1$ rondes (numérotées de 0 à $n_{eff} - 2$). Dans chaque ronde, $n_{eff}/2$ matchs sont joués simultanément. Chaque paire de joueurs $\{i, j\}$ (incluant le joueur fictif si n est impair) doit s'affronter exactement une fois au cours du tournoi. Pour chaque match, l'un des joueurs est désigné comme jouant "à domicile" (H) et l'autre "à l'extérieur" (A). L'objectif est de produire un calendrier complet qui minimise une fonction d'équité combinant plusieurs critères pour les n joueurs réels.

Entrées :

- n : Le nombre de joueurs réels (pair ou impair).

Sortie :

- Un calendrier complet spécifiant pour chaque ronde $r \in \{0, \dots, n_{eff} - 2\}$ et chaque match (i, j) qui joue à domicile (avec $i, j \in \{1, \dots, n_{eff}\}$).

2.2 Variables de Décision

2.2.1 Variable Principale

La décision fondamentale pour chaque match potentiel est de savoir qui joue à domicile. Nous utilisons une variable binaire principale pour cela :

$$x_{i,j,r} = \begin{cases} 1 & \text{si le joueur } i \text{ joue à domicile contre le joueur } j \text{ lors de la ronde } r \\ 0 & \text{sinon} \end{cases} \quad (1)$$

où $P_{eff} = \{1, \dots, n_{eff}\}$ est l'ensemble des participants et $R = \{0, \dots, n_{eff} - 2\}$ représente l'ensemble des rondes. L'indice du joueur correspond à son classement (1 étant le plus fort).

2.2.2 Variables Auxiliaires pour l'Équité

Pour modéliser les métriques d'équité, nous introduisons les variables auxiliaires suivantes, définies pour les n_{eff} participants :

- $is_home_{i,r}$: Variable binaire valant 1 si le participant $i \in P_{eff}$ joue à domicile lors de la ronde $r \in R$.
- $pen_seq_{i,r}$: Variable binaire valant 1 si le participant $i \in P_{eff}$ subit une "pénalité de séquence" (deux matchs consécutifs à domicile ou à l'extérieur) entre les rondes r et $r + 1$. Définie pour $r \in R' = \{0, \dots, n_{eff} - 3\}$.
- H_i : Variable entière représentant le nombre total de matchs joués à domicile par le participant $i \in P_{eff}$.
- $MaxDev$: Variable continue représentant l'écart absolu maximal entre le nombre de matchs à domicile d'un participant (H_i) et la moyenne idéale $\bar{H} = \frac{n_{eff}-1}{2}$.

2.3 Contraintes

2.3.1 Contraintes Fondamentales du Round Robin

- **Un match par participant par ronde** : Chaque participant doit jouer exactement un match lors de chaque ronde.

$$\sum_{j \in P_{eff}, j \neq i} x_{i,j,r} + \sum_{j \in P_{eff}, j \neq i} x_{j,i,r} = 1 \quad \forall i \in P_{eff}, \forall r \in R \quad (2)$$

- **Chaque paire se rencontre une fois** : Chaque paire de participants $\{i, j\}$ doit s'affronter exactement une fois au cours du tournoi.

$$\sum_{r \in R} (x_{i,j,r} + x_{j,i,r}) = 1 \quad \forall i, j \in P_{eff}, i < j \quad (3)$$

2.3.2 Contraintes d'Équité

Ces contraintes lient les variables auxiliaires aux variables de décision $x_{i,j,r}$. Elles sont définies pour les n_{eff} participants.

- **Lien $is_home_{i,r}$** : Détermine si un participant est à domicile.

$$is_home_{i,r} = \sum_{j \in P_{eff}, j \neq i} x_{i,j,r} \quad \forall i \in P_{eff}, \forall r \in R \quad (4)$$

- **Lien $pen_seq_{i,r}$ (Pénalités Séquence)** : Une pénalité se produit si $is_home_{i,r} = is_home_{i,r+1}$. Ceci est linéarisé comme suit :

$$pen_seq_{i,r} \geq is_home_{i,r} + is_home_{i,r+1} - 1 \quad \forall i \in P_{eff}, \forall r \in R' \quad (5)$$

$$pen_seq_{i,r} \geq (1 - is_home_{i,r}) + (1 - is_home_{i,r+1}) - 1 \quad \forall i \in P_{eff}, \forall r \in R' \quad (6)$$

où $R' = \{0, \dots, n_{eff} - 3\}$. Cette formulation capture les transitions H-H (première inéquation) et A-A (seconde inéquation). Chaque transition de ce type compte pour une pénalité.

- **Lien H_i (Total Domicile)** : Calcule le nombre total de matchs à domicile.

$$H_i = \sum_{r \in R} is_home_{i,r} \quad \forall i \in P_{eff} \quad (7)$$

- **Lien $MaxDev$ (Écart Max Domicile)** : Calcule l'écart maximal par rapport à la moyenne $\bar{H} = \frac{n-1}{2}$ (calculée uniquement sur les joueurs réels).

$$MaxDev \geq H_i - \bar{H} \quad \forall i \in P_{eff} \quad (8)$$

$$MaxDev \geq \bar{H} - H_i \quad \forall i \in P_{eff} \quad (9)$$

2.4 Métriques d'Équité

Les objectifs d'équité sont quantifiés par les métriques suivantes, que nous cherchons à minimiser (ou, dans le cas de HomeStrength, à amener proche de zéro). Ces métriques sont calculées sur le calendrier final pour les n joueurs réels, en ignorant les matchs impliquant le joueur fictif.

2.4.1 HomeStrength

Cette métrique évalue l'équilibre de la force des adversaires rencontrés par rapport au lieu du match (domicile/extérieur). Elle est définie comme la somme, sur tous les matchs entre joueurs réels, de la différence de rang entre l'adversaire et le joueur à domicile. En supposant que l'indice du joueur k (de 1 à n) représente son classement (1 étant le plus fort), la métrique est :

$$\text{HomeStrength} = \sum_{i=1}^n \sum_{j=1, j \neq i}^n \sum_{r=0}^{n_{eff}-2} \max(0, j - i) \cdot x_{i,j,r} \quad (10)$$

où $x_{i,j,r} = 1$ si le joueur réel i (domicile) joue contre le joueur réel j (extérieur) en ronde r . Cette métrique ne somme que les différences de rang positives ($j - i > 0$), c'est-à-dire uniquement lorsque le joueur à domicile (i) est mieux classé que son adversaire (j). Une valeur élevée indique que les joueurs forts reçoivent souvent des joueurs plus faibles à domicile. L'objectif est de minimiser cette valeur pour réduire cet avantage spécifique.

2.4.2 Pénalité de Séquence

Une "pénalité de séquence" est définie comme deux matchs consécutifs joués soit à domicile (H-H), soit à l'extérieur (A-A) pour un joueur réel. Le nombre total de pénalités est la somme des pénalités pour chaque joueur réel sur les $n_{eff} - 2$ transitions entre rondes :

$$\text{TotalPenalitesSequence} = \sum_{i=1}^n \sum_{r=0}^{n_{eff}-3} pen_seq_{i,r} \quad (11)$$

Minimiser cette somme favorise une alternance H/A plus régulière pour les joueurs réels.

2.4.3 MaxDeviation

Pour assurer une répartition homogène du nombre de matchs joués à domicile entre les n joueurs réels, nous cherchons à minimiser l'écart absolu maximal par rapport à la moyenne idéale. Le nombre idéal de matchs à domicile pour un joueur réel est $\frac{n-1}{2}$ si n est impair (car il

il y a n matchs contre des joueurs réels et un "bye") ou $\frac{n-1}{2}$ si n est pair (car il y a $n - 1$ matchs, dont la moitié à domicile en moyenne). La métrique MaxDeviation est calculée sur les comptes de matchs à domicile des n joueurs réels :

$$\text{MaxDeviation} = \max_{i=1,\dots,n} \left| \left(\sum_{r=0}^{n_{eff}-2} is_home_{i,r} \right) - \frac{n-1}{2} \right| \quad (12)$$

où la somme $\sum_{r=0}^{n_{eff}-2} is_home_{i,r}$ est le nombre total de matchs à domicile pour le joueur réel i .

2.5 Fonction Objectif

Nous combinons les trois métriques d'équité principales en une fonction objectif pondérée unique. Pour HomeStrength, l'objectif est d'avoir une valeur proche de zéro. Pour les autres, c'est la minimisation.

$$\text{Objectif} = f(\text{HomeStrength}, \text{TotalPenalitesSequence}, \text{MaxDeviation}) \quad (13)$$

La fonction exacte, incluant la pondération et la normalisation, est détaillée ci-dessous (Section ??).

2.6 Fonction Objectif Normalisée et Calibration des Poids Alpha et Beta

Les trois métriques d'équité – HomeStrength (HS, Équation ??), TotalPenalitesSequence (PS), et MaxDeviation (MD) – sont de natures et d'échelles intrinsèquement différentes. Une combinaison directe dans une fonction objectif pondérée nécessite une normalisation adéquate pour que les poids attribués à chaque critère reflètent fidèlement leur importance relative et pour que la recherche ne soit pas biaisée par les échelles disparates.

Contrairement à une normalisation empirique qui dépend d'un échantillon de solutions et peut varier, nous avons opté pour une **normalisation analytique**. Cette approche utilise les propriétés statistiques théoriques des métriques sous l'hypothèse d'une assignation aléatoire domicile/extérieur pour calculer une moyenne (μ) et un écart-type (σ) attendus pour chaque métrique. La métrique normalisée analytiquement (score Z) est alors définie comme :

$$M_k^{anal_norm} = \frac{M_k^{raw} - \mu_k}{\sigma_k} \quad (14)$$

2.6.1 Historique des approches de normalisation

Avant d'adopter la standardisation analytique, nous avons expérimenté deux autres méthodes :

1. Normalisation par les valeurs maximales. Cette première approche divisait chaque métrique par sa valeur maximale théorique ($M^{\max}$). Bien que séduisante par sa simplicité, elle faisait varier la notion même d'«équité» avec n : le ratio $M^{\max}/\mu$ double pour HS et PS et diverge pour MD, faussant les comparaisons entre tournois de taille différente.

2. Normalisation empirique. Nous avons ensuite essayé de centrer chaque métrique sur la moyenne et l'écart-type obtenus en simulant un grand nombre de calendriers aléatoires. Cette technique fonctionne pour de petites tailles ($n \lesssim 50$) mais devient prohibitive au-delà : le nombre de simulations nécessaires pour stabiliser les estimations de μ et σ croît au moins linéairement, et chaque simulation exige déjà $O(n^2)$ affectations.

3. Standardisation analytique. La méthode retenue dans ce rapport calcule μ et σ *exactement*, sans échantillonnage, à partir des distributions théoriques des métriques sous assignation domicile/extérieur aléatoire. Elle est indépendante de n , reproductible et garantit que deux tournois de tailles différentes sont comparés sur une base commune.

2.6.2 Échelle sigmoïde : motivation et interprétation

Bien que la minimisation directe de la somme pondérée des Z -scores (Éq. ??) suffise aux solveurs, ces scores peuvent dérouter les lecteurs : une valeur de -3 pour HS n'a pas la même «distance» intuitive qu'un $+3$ pour MD dès lors que les signes s'inversent et que les pondérations diffèrent. Pour retrouver une échelle uniforme dans $[0, 1]$, nous appliquons la fonction logistique

$$s(z) = \frac{1}{1 + e^{z/z_{\text{range}}}}, \quad z_{\text{range}} = 50, \quad (15)$$

où z est le Z -score d'une métrique. Deux raisons principales motivent ce choix :

1. **Bornage naturel.** La sigmoïde « écrase » toute valeur extrême dans la zone $(0, 1)$, évitant qu'une métrique domine indûment la somme pondérée.
2. **Interprétation probabiliste.** Dans le contexte du logit $\rightarrow$ probabilité $[0, 1]$, $s(z)$ peut se lire comme la probabilité qu'une assignation aléatoire produise une valeur plus défavorable que le calendrier étudié. Pour $z = 0$ on obtient $s(0) = 0.5$; un Z -score négatif ($z = -\sigma$) se traduit par $s \approx 0.73$, indiquant que le calendrier bat 73% des tirages aléatoires, et vice-versa.

Cette transformation est implémentée dans `normalization_manager.py` via la fonction `z_to_scaled` :

```
1 def z_to_scaled(z, z_range=50.0):
2     return 1 / (1 + math.exp(z / z_range))
```

Les solveurs minimisent ensuite la somme pondérée des versions sigmoïdes (variable `scaled_score`), ce qui rend les trois critères directement comparables : où M_k^{raw} est la valeur brute de la métrique $k \in \{\text{HS}, \text{PS}, \text{MD}\}$.

Les formules analytiques utilisées pour calculer μ et σ pour chaque métrique pour n joueurs réels sont les suivantes :

— **HomeStrength (HS) :**

$$\mu_{HS} = \frac{n(n-1)(n+1)}{12}$$

$$\sigma_{HS} = \frac{n\sqrt{n^2-1}}{4\sqrt{3}}$$

— **Penalty Sequence (PS) :** Pour n joueurs réels sur $n_{\text{eff}} - 1$ rondes, chaque joueur réel joue $n - 1$ matchs. Le nombre de transitions H/A est $n - 2$.

$$\mu_{PS} = n \cdot \frac{n-2}{2}$$

$$\sigma_{PS} = \frac{\sqrt{n(n-2)}}{2} \quad \text{si } n > 2, \text{ sinon } 1.0$$

— **Max Deviation (MD) :** Chaque joueur joue $n - 1$ matchs, avec $H_i \sim \text{Bin}(n - 1, 1/2)$. Le maximum parmi les n joueurs suit une distribution d'extrême valeur bien connue en statistique. Sous l'hypothèse d'indépendance (acceptable ici car chaque joueur a un

pattern propre), on peut approximer :

$$\mu_{MD} \approx \frac{\sqrt{n-1}}{2} \cdot \sqrt{2 \ln n}$$

$$\sigma_{MD} \approx \frac{\sqrt{n-1}}{2} \cdot \frac{\pi}{\sqrt{12 \ln n}}$$

Ces formules proviennent de l'approximation classique du maximum de n variables Gaussiennes centrées-réduites, adaptées ici au cas binomial avec $\mu = \frac{n-1}{2}$ et $\sigma = \frac{\sqrt{n-1}}{2}$. L'utilisation de ces expressions permet une mise à l'échelle cohérente des valeurs typiques de MaxDeviation sur tout n .

Dérivation des facteurs analytiques

Nous résumons ici, pour mémoire, les étapes menant aux expressions fermées de μ_k et σ_k sous l'hypothèse d'une assignation domicile/extérieur uniforme et indépendante.

HomeStrength (HS). Pour chaque paire $\{i, j\}$ ($i < j$) il y a exactement une rencontre et la probabilité que i reçoive j vaut $1/2$. Ainsi

$$\mathbb{E}[HS] = \frac{1}{2} \sum_{i=1}^{n-1} \sum_{j=i+1}^n (j - i) = \frac{n(n-1)(n+1)}{12}.$$

La variance s'obtient en additionnant des termes indépendants :

$$\text{Var}[HS] = \frac{n^2(n^2-1)}{48}, \quad \sigma_{HS} = \frac{n\sqrt{n^2-1}}{4\sqrt{3}}.$$

Penalty Sequence (PS). Pour un joueur donné il existe $R = n - 2$ transitions ; chacune engendre une pénalité avec probabilité $1/2$. Par la loi binomiale

$$\mathbb{E}[PS] = n \cdot \frac{R}{2} = \frac{n(n-2)}{2}, \quad \text{Var}[PS] = n \cdot \frac{R}{4}, \quad \sigma_{PS} = \frac{\sqrt{n(n-2)}}{2}.$$

Max Deviation (MD). Soit $H_i \sim \text{Bin}(n-1, \frac{1}{2})$. Le maximum $M_n = \max_i |H_i - \frac{n-1}{2}|$ est asymptotiquement Gumbel ; on obtient

$$\mu_{MD} \approx \frac{\sqrt{n-1}}{2} \sqrt{2 \ln n}, \quad \sigma_{MD} \approx \frac{\sqrt{n-1}}{2} \frac{\pi}{\sqrt{12 \ln n}}.$$

Ces expressions coïncident avec l'implémentation Python. La normalisation analytique présente l'avantage d'être indépendante d'un échantillon particulier de calendriers, fournissant une base de comparaison cohérente entre différentes exécutions ou méthodes. Les scores Z résultants $M_k^{\text{anal_norm}}$ indiquent de combien d'écarts-types la valeur brute s'écarte de la moyenne théorique attendue sous assignation aléatoire. Des valeurs négatives sont préférables car elles indiquent une performance meilleure que la moyenne aléatoire.

Les solveurs (MILP et SA) cherchent à minimiser une combinaison pondérée de ces métriques normalisées analytiquement. La fonction objectif est formulée comme la somme pondérée des scores Z :

$$Z = \text{anal_norm_hs} + \alpha \cdot \text{anal_norm_ps} + \beta \cdot \text{anal_norm_md} \quad (16)$$

Les paramètres $\alpha \geq 0$ et $\beta \geq 0$ sont les poids qui déterminent l'importance relative de la pénalité de séquence et de la déviation maximale par rapport à HomeStrength.

Quantification de l'écart entre valeurs maximales et valeurs typiques Il est instructif de comparer les écarts relatifs entre la valeur maximale possible d'une métrique et sa valeur moyenne typique sous assignation aléatoire :

- **HomeStrength** : $\mu_{HS} = \frac{n(n-1)(n+1)}{12}$, $S_{max} = \frac{n(n-1)(n+1)}{6}$. Donc $S_{max} = 2 \cdot \mu_{HS}$, et ce ratio est constant quel que soit n .
- **Penalty Sequence** : $\mu_{PS} = \frac{n(n-2)}{2}$, $PS_{max} = n(n-2)$ donc même ratio 2.
- **MaxDeviation** : $\mu_{MD} \approx \frac{\sqrt{n-1}}{2} \cdot \sqrt{2 \ln n}$, $MD_{max} = \frac{n-1}{2}$ donc le ratio est :

$$\frac{MD_{max}}{\mu_{MD}} = \frac{\frac{n-1}{2}}{\frac{\sqrt{n-1}}{2} \sqrt{2 \ln n}} = \sqrt{\frac{n-1}{2 \ln n}},$$

ce ratio croît comme $\sqrt{n/\ln n}$ et diverge lentement avec n .

Cela signifie que la normalisation par le maximum devient de plus en plus biaisée à mesure que n augmente, surtout pour MaxDeviation. La normalisation analytique permet de corriger ce biais de manière rigoureuse.

3 Calibration des Poids et Analyse du Front de Pareto

La calibration des poids α et β dans la fonction objectif (Équation ??) est cruciale pour naviguer les compromis inhérents entre les trois objectifs d'équité (HomeStrength, Pénalité de Séquence, Max Deviation), une fois ceux-ci normalisés analytiquement. Pour ce faire, nous avons exécuté notre algorithme de Recuit Simulé optimisé pour $n = 500$ joueurs, en variant systématiquement les paires (α, β) .

Chaque exécution avec une paire (α, β) spécifique produit un calendrier, dont les trois métriques normalisées (Z_{HS} , Z_{PS} , Z_{MD}) constituent un point dans l'espace tridimensionnel des objectifs. L'ensemble de ces points forme un nuage à partir duquel le front de Pareto est extrait. Ce front regroupe les solutions non-dominées : celles pour lesquelles aucune amélioration sur un critère n'est possible sans en dégrader au moins un autre. La Figure ?? illustre ce front. Les scores Z , indiquant l'écart par rapport à la moyenne théorique en termes d'écarts-types, sont préférentiellement négatifs, signifiant une performance meilleure que la moyenne aléatoire.

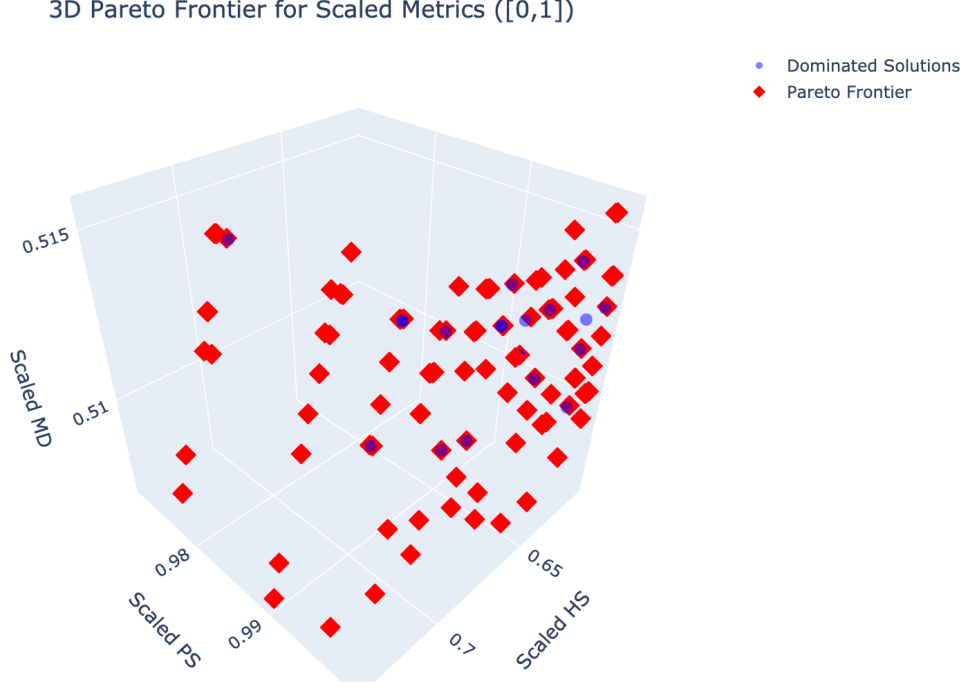


FIGURE 1 – Front de Pareto 3D des métriques d'équité normalisées (Z_{HS} , Z_{PS} , Z_{MD}) pour $n = 500$. Les points rouges sont Pareto-optimaux. Des valeurs plus faibles (négatives) sont meilleures.

Le choix d'une configuration (α, β) spécifique sur ce front dépend des priorités accordées à chaque critère. Notre analyse pour $n = 500$ a conduit à la sélection de $\alpha = 0.8$ pour la Pénalité de Séquence et $\beta = 1.2$ pour la Max Deviation, avec un poids implicite de 1 pour HomeStrength. Ce choix offre un bon équilibre, produisant des Z-scores négatifs pour les trois métriques, ce qui est désirable.

La légère prépondérance accordée à Max Deviation ($\beta = 1.2$) par rapport à la Pénalité de Séquence ($\alpha = 0.8$) et HomeStrength (poids de 1) est délibérée. La métrique HomeStrength, si elle était le seul critère ou un critère trop dominant, pourrait potentiellement favoriser des calendriers où les joueurs forts jouent souvent à l'extérieur contre d'autres joueurs forts (pour minimiser $\sum \max(0, j - i) \cdot x_{i,j,r}$ lorsque i et j sont forts et $j - i$ est petit ou négatif), ce qui pourrait indirectement déséquilibrer la répartition globale des matchs à domicile. En augmentant légèrement le poids de Max Deviation, nous visons à assurer une distribution plus homogène des matchs à domicile pour tous les joueurs, contrant ainsi d'éventuels effets secondaires indésirables d'une focalisation excessive sur HomeStrength. Ces poids, une fois déterminés, ont été utilisés de manière cohérente pour toutes les évaluations comparatives des algorithmes.

Note pour l'exploration interactive : Une version interactive de ce front de Pareto est disponible. Après avoir cloné le dépôt GitLab du projet, exécutez la commande suivante dans le terminal depuis la racine du projet :

```
open calibration_plots_n500_analytical_norm/pareto_3d_interactive_n500_anal_norm.html
```

4 Développement et Performance du Recuit Simulé

4.1 Protocole Expérimental et Stratégie d'Optimisation

Pour évaluer l'efficacité de notre méta-heuristique de Recuit Simulé (SA), nous avons adopté une approche itérative. Initialement, une version "naïve" du SA (implémentée dans `sa_solver_non_opti.py`) a été développée. Cette version, bien que fonctionnellement correcte, recalculait intégralement les

métriques à chaque itération et utilisait des structures de données Python standard, la rendant lente pour des instances de grande taille.

Conscients de cette limitation, nous avons rapidement entrepris d'optimiser significativement l'algorithme (version `sa_solver.py`). Les optimisations clés incluent :

- **Compilation Just-in-Time (JIT) avec Numba** : La boucle principale du SA et les fonctions critiques de calcul de métriques ont été décorées avec `@numba.njit`, permettant une compilation en code machine quasi-natif.
- **Utilisation de NumPy** : Les calendriers et les compteurs ont été représentés par des tableaux NumPy, favorisant des opérations vectorielles rapides.
- **Mises à jour incrémentielles** : Plutôt que de recalculer entièrement les métriques (HomeStrength, Pénalité de Séquence, Max Deviation) après chaque mouvement (inversion domicile/extérieur d'un match), nous avons implémenté des mises à jour différentielles. Cela réduit considérablement le coût de chaque itération.
- **Représentation compacte des séquences** : Le module `packed_array_utils.py` utilise un encodage binaire pour stocker les statuts domicile/extérieur (H/A) de plusieurs joueurs dans un seul octet, optimisant l'accès mémoire et le calcul des pénalités de séquence.
- **Simplification des Voisinages** : La version optimisée se concentre sur un seul type de mouvement (inversion H/A d'un match aléatoire), tandis que la version non optimisée explorait d'autres mouvements plus complexes (échange de rondes, échange de joueurs). Cette simplification facilite grandement l'implémentation de mises à jour incrémentielles efficaces.
- **Parallélisation des *runs* multiples** : L'utilisation de `concurrent.futures.ProcessPoolExecutor` permet de lancer plusieurs chaînes de SA indépendantes en parallèle, exploitant les cœurs multiples des processeurs modernes pour une meilleure exploration de l'espace des solutions.

Cette stratégie d'optimisation précoce a eu des conséquences importantes. D'une part, elle nous a permis d'évaluer le comportement du SA sur des instances de très grande taille ($N \geq 500$) bien plus tôt dans le projet. Cette capacité a été cruciale pour identifier les limites de notre approche initiale de normalisation des métriques (basée sur les valeurs maximales), qui se comportait mal pour N élevé. Cela nous a conduits à adopter la normalisation analytique (basée sur les Z-scores), plus robuste et cohérente.

D'autre part, la complexité inhérente au code optimisé (en particulier avec Numba et les mises à jour incrémentielles) a rendu certaines adaptations ultérieures plus laborieuses. Un exemple notable est l'implémentation d'un budget temps strict pour les comparaisons. Initialement, notre SA optimisé fonctionnait avec un nombre fixe d'itérations. Pour permettre une comparaison équitable avec d'autres approches sur la base d'un temps d'exécution fixe, nous avons dû intégrer un mécanisme de gestion du temps :

1. Une phase de "warm-up" (ou "test d'étalonnage") est exécutée au début : l'algorithme effectue un petit nombre d'itérations pour mesurer la vitesse d'itération sur la machine actuelle.
2. Le nombre total d'itérations pour la boucle principale du SA est ensuite ajusté pour tenter de respecter le budget temps imparti, tout en assurant un refroidissement progressif de la température.

Bien que fonctionnel, ce mécanisme n'est pas infallible. L'estimation de la vitesse d'itération peut être optimiste, surtout lorsque plusieurs *runs* parallèles saturant le CPU, pouvant entraîner un phénomène de "throttling" (ralentissement thermique du processeur). Ainsi, le temps d'exécution final peut parfois légèrement dépasser le budget alloué.

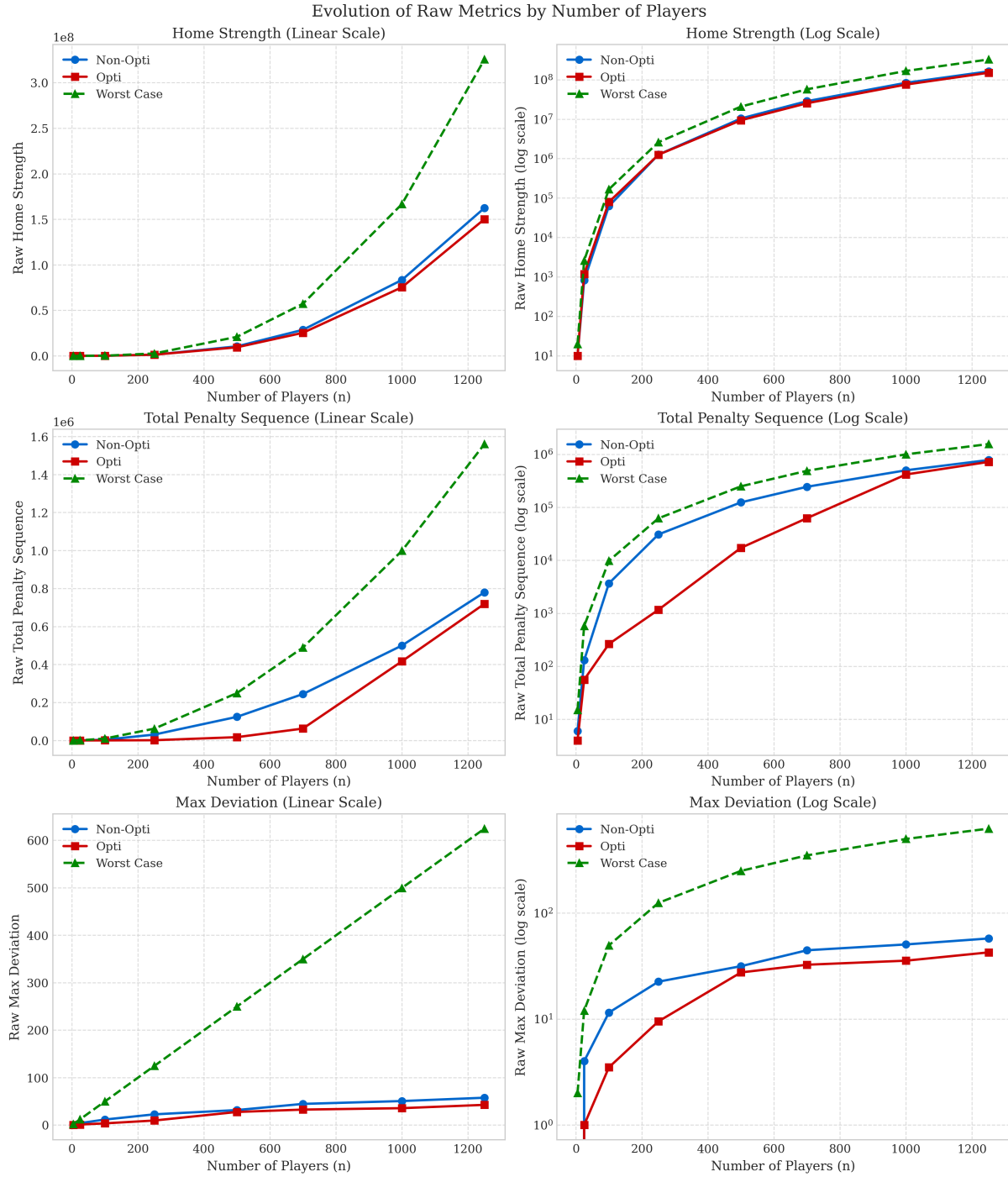
4.2 Résultats Chiffrés des Comparaisons SA

Chaque version du SA a été lancée une fois pour différentes tailles n , avec une limite de temps de 60 secondes. Les poids pour la fonction objectif étaient $\alpha = 0.8, \beta = 1.2$.

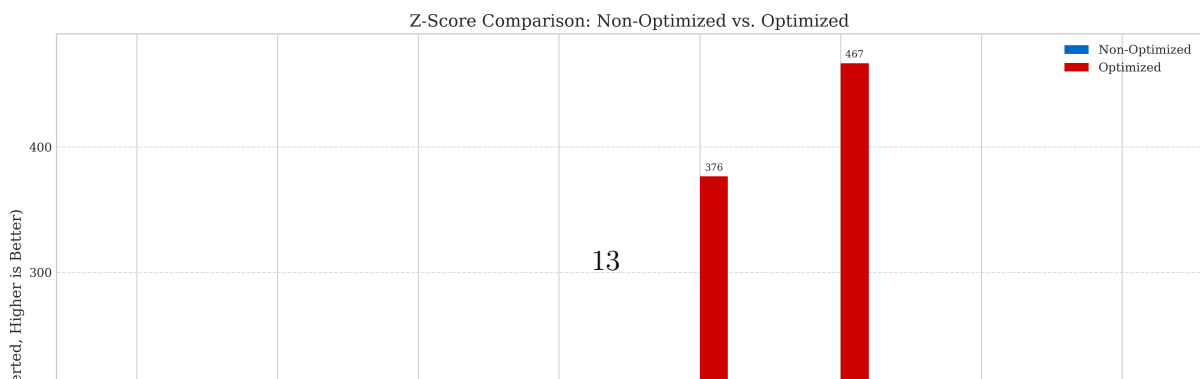
TABLE 1 – Comparaison des performances (métriques brutes et Z-sum) entre les versions SA Non-Optimisée et Optimisée (60s de budget temps). Z-sum : plus il est faible (négatif), mieux c'est. Métriques brutes : plus elles sont faibles, mieux c'est.

n	Z-sum		HomeStrength (brut)		Penalty Seq (brut)		Max Deviation (brut)	
	Non-Opti	Opti	Non-Opti	Opti	Non-Opti	Opti	Non-Opti	Opti
5	-3.6314	-4.4576	10	10	6	4	0.00	0.00
25	-17.9921	-21.7879	819	1184	130	56	4.00	1.00
100	-36.4179	-84.2157	62 233	79 476	3678	264	11.50	3.50
250	-7.2309	-203.7024	1 261 646	1 253 161	30 800	1168	22.50	9.50
500	-4.3434	-376.4995	10 397 292	9 372 697	124 038	17 196	31.50	27.50
700	-1.6844	-466.7747	28 561 754	25 290 130	244 072	62 438	44.50	32.50
1000	-0.9371	-192.9358	83 349 223	75 316 457	499 478	416 408	50.50	35.50
1250	-3.7943	-138.7044	162 424 383	150 166 456	779 642	719 098	57.50	42.50

4.3 Analyse Visuelle et Qualitative des Plannings



(a) Évolution des valeurs brutes de HomeStrength (HS), Penalty Sequence (PS) et Max Deviation (MD) en fonction de n pour les versions *Non-Opti* (bleu) et *Opti* (rouge) du Recuit Simulé, comparées au pire cas théorique (vert). Chaque point correspond à la meilleure solution trouvée en 60 s.



La Figure ?? et la Figure ?? confirment la supériorité de la version optimisée du SA. Concernant le Z-sum (Fig. ??), le pic de performance est atteint à $n = 700$ pour SA-Opti, tandis que SA-NonOpti plafonne vers $n = 100$. Cette "dégradation" apparente du Z-score pour N très grands sous budget temps fixe s'explique par l'immensité croissante de l'espace de recherche et la difficulté à atteindre des améliorations absolues proportionnellement aussi importantes que pour des N intermédiaires.

4.3.1 Qualité d'un Planning Exemple ($n = 25$)

Le planning généré par SA-Opti pour $n = 25$ (consultable dans le fichier `sa_schedule_n25.csv` produit par le code) illustre la qualité atteignable. Le nombre de matchs à domicile par joueur y est très bien équilibré : l'idéal est $(25 - 1)/2 = 12$. Les valeurs observées dans l'exemple de fichier CSV fourni varient entre 11 et 13, résultant en un Max Deviation de 1.0, ce qui est excellent.

Extrait des séquences H/A pour $n = 25$ (issues du fichier `sa_schedule_n25.csv`) pour deux joueurs :

- Joueur 1 : HAAAAHAHAHAHAHAHAHAH
- Joueur 25 : HAHHAHAHAHAHAHAHAHAH

Ces séquences montrent une bonne alternance générale. On observe occasionnellement des séries de 3 matchs consécutifs à domicile ou à l'extérieur (par ex., Joueur 1 : AAA). Une série de 4 (AAAA ou HHHH) est plus rare mais peut survenir. Notre définition actuelle de la Pénalité de Séquence (Éq. (5) et (6)) pénalise chaque transition H-H ou A-A d'un point. Ainsi, HHH compte pour deux pénalités (H1-H2 et H2-H3), et HHHH pour trois. Pour renforcer la lutte contre les longues séries, une fonction de pénalité non-linéaire (e.g., coût de $(L - 1)^2$ pour une série de longueur L) pourrait être explorée, bien que cela complexifierait les mises à jour incrémentielles.

5 Conclusion et Perspectives

Ce projet a permis de développer et d'évaluer une approche basée sur le Recuit Simulé pour la génération de calendriers de tournois Round Robin équitables. L'optimisation du SA, combinée à une normalisation analytique robuste des métriques d'équité et une calibration soignée des poids de la fonction objectif, a démontré sa capacité à produire des solutions de haute qualité, même pour des instances de grande taille, dans un temps de calcul contraint.

Les comparaisons avec une version non optimisée ont validé l'efficacité de notre SA optimisé. L'analyse des plannings générés montre un bon respect des différents critères d'équité, bien que des améliorations ciblées (par exemple, sur la pénalisation des longues séries H/A) soient toujours possibles.

Pour des travaux futurs, l'exploration d'autres méta-heuristiques (comme la recherche Tabou initialement envisagée) ou l'hybridation d'approches pourrait s'avérer fructueuse. De même, l'étude de fonctions de pénalité alternatives pour la métrique de séquence pourrait affiner davantage l'équité des calendriers sur cet aspect spécifique.

Instructions pour la Reproductibilité et Exploration

Le code source complet de ce projet est disponible sur un dépôt GitLab (le lien sera fourni séparément). Le dépôt contient :

- Les implémentations des solveurs (e.g., `sa_solver.py`, `sa_solver_non_opti.py`).
- Les scripts pour générer les données de calibration et les benchmarks.
- Un fichier `README.md` détaillé avec les instructions pour installer les dépendances, exécuter les solveurs, et reproduire les résultats présentés dans ce rapport.
- Un `Makefile` simplifiant certaines commandes courantes.

Nous encourageons le lecteur à cloner le dépôt et à explorer le code et les outils fournis pour une compréhension plus approfondie de notre travail.